

Rapport TP2 - Gestion des processus

Module : Système d'exploitation 2

Enseignante : Y. BEN JMAA

Année Universitaire : 2025/2026

Introduction

Ce rapport présente les solutions et analyses des exercices du TP2 sur la gestion des processus en C utilisant l'appel système `fork()`. Les objectifs étaient de maîtriser la création de processus, la synchronisation et la communication entre processus.

1 Exercice 1 - Programme fork1.c

1.1 Résultat d'exécution

```
hello je programme
hello je programme
```

1.2 Explication

Après l'appel système `fork()`, le processus se divise en deux :

- Le **processus père** et le **processus fils** s'exécutent concurremment
- Chaque processus exécute l'instruction `printf("hello je programme\n")`
- Le message s'affiche donc **deux fois**, mais l'ordre peut varier selon l'ordonnement du système
- Les deux processus terminent avec `return(0)`

2 Exercice 2 - Programme fork2.c

2.1 Résultat d'exécution typique

```
1234 père de 1235
1235 fils de 1234
```

2.2 Explication

1. Le père exécute `fork()` et obtient le PID du fils (> 0)
2. Le fils obtient la valeur 0 de `fork()`
3. Dans le fils :
 - Affichage de son PID (`getpid()`) et du PID du père (`getppid()`)

- Terminaison avec `exit(7)`
- 4. Dans le père :
 - Affichage de son PID et du PID du fils
 - Appel à `wait()` pour attendre la terminaison du fils
 - Le statut de terminaison (7) n'est pas récupéré car `wait()` est sans paramètre

3 Exercice 3 - Programme fork3.c

3.1 Analyse du nombre de processus

Le programme crée **4 processus** au total :

Processus initial (P0)

```
fork() → Crée P1
    fork() → Crée P3 (depuis P1)
    (P1 continue)
fork() → Crée P2 (depuis P0)
    fork() → Crée P4 (depuis P2)
```

3.2 Explication

1. Premier `fork()` : $P0 \rightarrow P1$ (2 processus)
2. Chacun exécute le deuxième `fork()` :
 - P0 crée P2
 - P1 crée P3
 - P2 exécute aussi le deuxième `fork()` et crée P4
3. Tous les processus s'arrêtent sur `pause()`

4 Exercice 4 - Programme fork4.c

4.1 Version initiale (non synchronisée)

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main() {
7     pid_t pid1, pid2;
8     int i;
9
10    pid1 = fork();
11    if (pid1 == 0) {
12        for (i = 1; i <= 50; i++) {
13            printf("%d_", i);
14        }
15        printf("\n");
16        exit(0);
17    } else {
```

```
18     pid2 = fork();
19     if (pid2 == 0) {
20         for (i = 51; i <= 100; i++) {
21             printf("%d_", i);
22         }
23         printf("\n");
24         exit(0);
25     }
26 }
27 wait(NULL);
28 wait(NULL);
29 return 0;
30 }
```

4.2 Remarque sur l'exécution

Les affichages des deux processus fils sont **entrelacés** (mêlés) car ils s'exécutent en parallèle sans synchronisation.

4.3 Version synchronisée (affichage ordonné)

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4  #include <stdlib.h>
5
6  int main() {
7      pid_t pid1, pid2;
8      int i;
9
10     pid1 = fork();
11     if (pid1 == 0) {
12         for (i = 1; i <= 50; i++) {
13             printf("%d_", i);
14         }
15         printf("\n");
16         exit(0);
17     }
18
19     wait(NULL); // Attendre la fin du premier fils
20
21     pid2 = fork();
22     if (pid2 == 0) {
23         for (i = 51; i <= 100; i++) {
24             printf("%d_", i);
25         }
26         printf("\n");
27         exit(0);
28     }
29
30     wait(NULL); // Attendre la fin du deuxième fils
31     return 0;
32 }
```

5 Exercice 5 - Analyse d'un programme complet

5.1 1. Compilation et exécution normale

Séquence d'affichage :

```
Le père démarre
Je suis le père
Je suis en attente de mon fils
Je suis le fils
Mon PID est: [PID_fils]
Je vais dormir pour 10 secondes
Mon fils a terminé
```

Interprétation :

- Le père attend avec `wait()` la terminaison du fils
- Le fils dort 10 secondes puis se termine
- Le père reprend et affiche le message final

5.2 2. Exécution en arrière-plan et `ps -f`

Commande :

```
./f5 &
ps -f
```

Résultat de `ps -f` :

- Pendant le sommeil du fils : deux processus actifs (père en attente, fils en sommeil)
- Le père est dans l'état **S** (sleeping) à cause de `wait()`
- Le fils est dans l'état **S** à cause de `sleep(10)`

5.3 3. Suppression de `wait()`

Observation :

- Le père termine immédiatement sans attendre le fils
- Le fils continue son exécution pendant 10 secondes
- Le message "Mon fils a terminé" s'affiche **avant** la fin réelle du fils
- Le fils devient **zombie** jusqu'à sa terminaison

5.4 4. Programme sans `wait()` en arrière-plan avec `ps -f`

Observation :

- Si le père termine avant le fils : le fils apparaît comme **zombie** (état **Z**)
- Le processus zombie reste dans la table des processus jusqu'à ce que le père appelle `wait()` ou termine

5.5 5. Ajout d'une variable globale

Code modifié :

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int variable_globale = 0;
6
7 int main() {
8     pid_t rf;
9     printf("Le_p re_marre\n");
10    printf("Variable_globale_initiale: %d\n", variable_globale);
11
12    rf = fork();
13
14    if (rf > 0) {
15        variable_globale = 100;
16        printf("P re_Variable_globale_modifie: %d\n",
17              variable_globale);
18        wait(NULL);
19        printf("P re_Variable_globale_finale: %d\n", variable_globale);
20    }
21    else if (rf == 0) {
22        variable_globale = 200;
23        printf("Fils_Variable_globale_modifie: %d\n",
24              variable_globale);
25        sleep(2);
26        printf("Fils_Variable_globale_avant_termination: %d\n",
27              variable_globale);
28    }
29    else {
30        printf("Erreur d'ex cution de fork\n");
31    }
32
33    return 0;
34 }
```

Résultat et interprétation :

Le père démarre

Variable globale initiale: 0

Père - Variable globale modifiée: 100

Fils - Variable globale modifiée: 200

Fils - Variable globale avant terminaison: 200

Père - Variable globale finale: 100

Explication : Après `fork()`, le père et le fils ont des **copies séparées** de la mémoire :

- Modification dans le fils (200) n'affecte pas la variable du père
- Modification dans le père (100) n'affecte pas la variable du fils
- Chaque processus travaille sur sa propre copie de la variable globale

Conclusion générale

Ce TP a permis de comprendre :

1. Le mécanisme de création de processus avec `fork()`
2. La différenciation père/fils via la valeur de retour
3. La nécessité de la synchronisation avec `wait()`
4. L'indépendance des espaces mémoire entre processus
5. Les états des processus (actif, zombie, etc.)

Les concepts maîtrisés sont essentiels pour la programmation système et la gestion des processus concurrents.